K L (DEEMED TO BE) UNIVERSITY / COMPUTER SCIENCE & ENGINEERING

Parallel File Search Using Threads

A multithreaded file-search application built with C++17 and Qt 6

PROJECT TEAM

D Pawan Kumar · 2520090102
Ajith Reddy · 2520090186
Hitesh Suthar · 2520090078

FACULTY GUIDE

Raghupathi M

Hyderabad · Academic Year 2026–2027

A practical application of concurrency

A desktop tool to search filenames and text across directory trees.

THE NEED

Large folders make manual and sequential file inspection time-consuming.

THE APPROACH

A coordinator schedules independent file-search tasks to a reusable worker pool.

THE OUTCOME

A responsive GUI with progress information, result filtering, activity logs, and CSV export.

Core principle: keep interface work separate from concurrent file processing.

Built for developers, administrators, students, researchers, and everyday computer users.

The problem is more than search speed

The system addresses performance, responsiveness, and coordination challenges.

01 SLOW INSPECTION

Sequential scans inspect files one by one and can be slow across large directory trees.

02 UI BLOCKING

Running file operations on the GUI thread can make the application appear unresponsive.

03 CONCURRENCY RISKS

Multiple workers require safe shared-state updates, cancellation, progress reporting, and clean shutdown.

Problem definition: build a concurrent search tool that remains usable while managing threads and filesystem resources safely.

A focused, useful feature set

Features documented in the project report.

SEARCH

Filename, file contents, or both

MATCHING

Case options and regular expressions

TRAVERSAL

Recursive or immediate directory entries

FILE SAFETY

Binary detection; skip files over 50 MB

MONITORING

Progress, active workers, elapsed time, logs

RESULTS

Sort, quick-filter, context actions, CSV export

Innovation through thoughtful engineering

The project combines established technologies in a practical, integrated workflow.

BOUNDED, REUSABLE THREAD POOL

Reuses worker threads instead of creating one thread per file; schedules batches of 25 paths per task.

RESPONSIVE UI SEPARATION

Search processing is separated from the Qt interface; progress is relayed through signals and slots.

RESOURCE-AWARE SEARCH

Reads content line by line and skips files larger than 50 MB to constrain per-file processing.

USABLE SEARCH OUTPUT

Combines regex options, binary detection, cancellation, activity logs, and CSV export.

Novelty claim is based on the documented design—not a claim of being the first such tool.

Operating-system ideas in action

The project makes concurrency and synchronization visible in a real application.

THREADS

Worker threads process independent batches while sharing process resources.

MUTEX

Protects the task queue and shared result collections.

CONDITION VARIABLE

Workers wait efficiently until tasks arrive or shutdown begins.

ATOMIC STATE

Tracks running/cancellation flags and progress counters.

FILESYSTEM

std::filesystem traverses directories; std::ifstream reads lines.

LIFECYCLE

Owned threads are joined during clean shutdown.

A modular pipeline with clear responsibilities

GUI, coordination, worker execution, logging, and export are separated.



LOGGER

Timestamped INFO, SUCCESS, WARN, and ERROR messages.

EXPORT MANAGER

Escaped CSV output with UTF-8 BOM encoding.

DATA MODELS

SearchResult and SearchOptions represent results and settings.

Qt signals and slots carry updates back to the UI thread.

From user query to useful results

The coordinator discovers files and distributes independent work.

01 **Choose directory**

02 **Enter pattern**

03 **Configure options**

04 **Discover files**

05 **Batch paths**

06 **Schedule tasks**

07 **Match filenames / text**

08 **Update safe counters**

09 **Review and filter**

10 **Export CSV**

On completion or cancellation, the application coordinates shutdown and joins owned threads.

Implementation choices that matter

Focused engineering details behind the user-facing features.

TASK SCHEDULING

Mutex-protected queue, condition-variable notifications, futures for task results.

CONTENT MATCHING

Incremental line-by-line reads; ordinary text and std::regex matching.

FILE HANDLING

Heuristic binary detection and a 50 MB maximum file-size guard.

CANCELLATION

Atomic flags communicate cancellation and running state.

SYNCHRONIZATION

Mutexes protect shared results; critical sections should stay short.

BUILD & TEST

CMake targets the core library, desktop app, and CTest test suite.

The actual application

Screenshots reproduced from the supplied project report, pages 13–14.

CHAPTER 7 — EXECUTION AND DEMONSTRATION

7.1 Build Procedure

Install Qt 6 and a compatible C++17 toolchain, then run these commands from the project root:

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build --config Release
ctest --test-dir build --output-on-failure
```

The exact launch path depends on the generator and configuration. A clean build on the submission machine is recommended.

7.2 User Workflow

9. Launch ParallelFileSearch.
10. Choose the target directory.
11. Enter a text or regular-expression pattern.
12. Choose filename-only, content-only, or combined mode.
13. Set recursion, case sensitivity, regex behavior, and worker count.
14. Start searching and observe progress and activity logs.
15. Filter or sort the results table.
16. Open or copy a result, or export results to CSV.

7.3 Application Screenshots

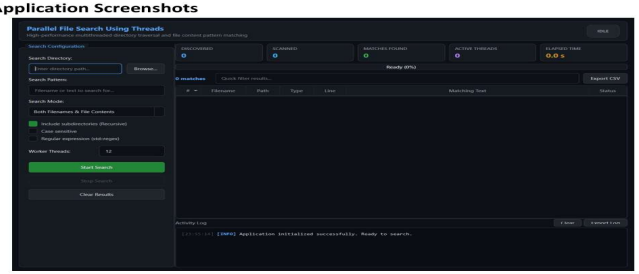


Figure 7.1: Application interface and search configuration.

SEARCH CONFIGURATION

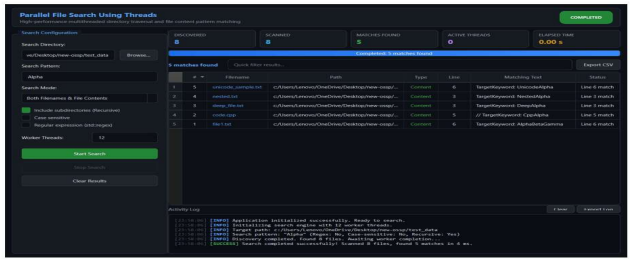


Figure 7.2: Results dashboard after a search.

7.4 Test Plan

Test target	Coverage focus
Thread pool	Task enqueueing, execution, futures, worker lifecycle
Search engine	Pattern matching, search modes, directory traversal
Cancellation	Cancellation request and clean completion
CSV export	Field escaping and UTF-8 BOM output

The project contains corresponding test source files and CTest configuration. This report does not claim that all tests passed because test execution results have not been independently verified.

RESULTS DASHBOARD

A documented test plan—not assumed results

The report identifies these test targets; outcomes must be verified by running the suite.

THREAD POOL

Task enqueueing, execution, futures, and worker lifecycle.

SEARCH ENGINE

Pattern matching, search modes, and directory traversal.

CANCELLATION

Cancellation request and clean completion.

CSV EXPORT

Field escaping and UTF-8 BOM output.

Validation step: run ctest --test-dir build --output-on-failure and record the actual output.

Make the project easy to reproduce

Documentation is strongest when another person can build, run, and verify the system.

README

Project overview, prerequisites, setup, and launch instructions.

ARCHITECTURE

Module responsibilities, workflow, and concurrency design.

USER GUIDE

Search configuration, result handling, and CSV export steps.

BUILD STEPS

CMake configure/build commands for the target environment.

TEST EVIDENCE

CTest output and reproducible test cases should be added after execution.

SCREENSHOTS

Interface and results screenshots from the project report.

Who benefits—and how the team collaborates

Use the team contribution fields below to reflect the work actually completed.

D PAWAN KUMAR · 2520090102

Contribution: [add actual module / responsibility]

AJITH REDDY · 2520090186

Contribution: [add actual module / responsibility]

HITESH SUTHAR · 2520090078

Contribution: [add actual module / responsibility]

POTENTIAL BUSINESS IMPACT

Can help developers search code, administrators inspect logs, and users locate information across folders.

Before presenting: replace contribution placeholders with verified team responsibilities.

A practical demonstration of operating-system design

Concurrency is valuable when paired with clear architecture, safe coordination, and usable output.

TAKEAWAY

A Qt interface and reusable C++17 search core demonstrate threads, synchronization, filesystem traversal, cancellation, and lifecycle management.

GITHUB REPOSITORY

Add your real repository URL here:

[PASTE GITHUB REPOSITORY LINK]

Repository URL was not supplied, so no link has been invented.

REFERENCES

C++ reference: en.cppreference.com · Qt: doc.qt.io

CMake: cmake.org/cmake/help/latest · RFC 4180: rfc-editor.org/rfc/rfc4180

THANK YOU · QUESTIONS